

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab_03_)

Apply Behavioral-level Verilog to Design and Simulate Combinational Modules

Student Name: ___Areeb Fatima___ Roll No: _____24I-6543_____ Section: _B_

Date performed: _____2nd September_____, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Task 1:

Description:

In this task, we were asked to make the adder/subtractor, so firstly we implement it by Gate level modeling by using the half and full adders for four bits. After that, we apply the Behavioral level in which we make the ctrl input, as if ctrl is 0 then addition will perform else subtraction by using if-else command. After that, we also make the Logisim circuit and 2 4-bit inputs with ctrl extend to 4 bits and XOR with B for subtraction.

Code:

4-bit Adder (Gate Level):

```
module half_adder(A,B,sum,carry);  
  
input A,B;  
  
output sum,carry;  
  
xor(sum,A,B);  
  
and(carry,A,B);  
  
endmodule  
  
module full_adder(a,b,CIN,SUM,CARRY);  
  
input a,b,CIN;  
  
output SUM,CARRY;  
  
wire t1,t2,t3;  
  
half_adder ha1(a,b,t1,t2);  
  
half_adder ha2(t1,CIN,SUM,t3);  
  
or o1(CARRY,t2,t3);  
  
endmodule  
  
module four_bitAdder(A,B,S,cout);  
  
input [3:0] A,B;
```

```
output [3:0] S;

output cout;

wire t1,t2,t3;

half_adder u1(A[0],B[0],S[0],t1);

full_adder u2(A[1],B[1],t1,S[1],t2);

full_adder u3(A[2],B[2],t2,S[2],t3);

full_adder u4(A[3],B[3],t3,S[3],cout);

endmodule

module tb_adder();

reg [3:0] x,y;

wire [3:0] sum;

wire cout;

four_bitAdder uut(x,y,sum,cout);

initial

begin

x=4'b1001; y=4'b1010; //5+10

#50

x=4'b0000; y=4'b1001; //0+9

#50

x=4'b1111; y=4'b0010; //15+2

#50

x=4'b0110; y=4'b0001; //6+1

end
```

```
endmodule
```

4-bit Adder (Behavioral Level):

```
module four_bitAdder(A,B,ctrl,S);
```

```
input [3:0] A,B;
```

```
input ctrl;
```

```
output reg [3:0] S;
```

```
always @(*)
```

```
begin
```

```
if(ctrl==0)
```

```
S= A+B;
```

```
else
```

```
S=A-B;
```

```
end
```

```
endmodule
```

```
module tb_adder();
```

```
reg [3:0] x,y;
```

```
reg ctrl;
```

```
wire [3:0] sum;
```

```
four_bitAdder uut(x,y,ctrl,sum);
```

```
initial
```

```
begin
```

```
ctrl= 0; x=4'b1001; y=4'b1010; //5+10
```

```
#50
```

```
ctrl= 1; x=4'b0000; y=4'b1001; //0+9
```

```
#50
```

```
ctrl= 1; x=4'b1111; y=4'b0010; //15+2
```

```
#50
```

```
ctrl= 0; x=4'b0110; y=4'b0001; //6+1
```

```
end
```

```
endmodule
```

Output:

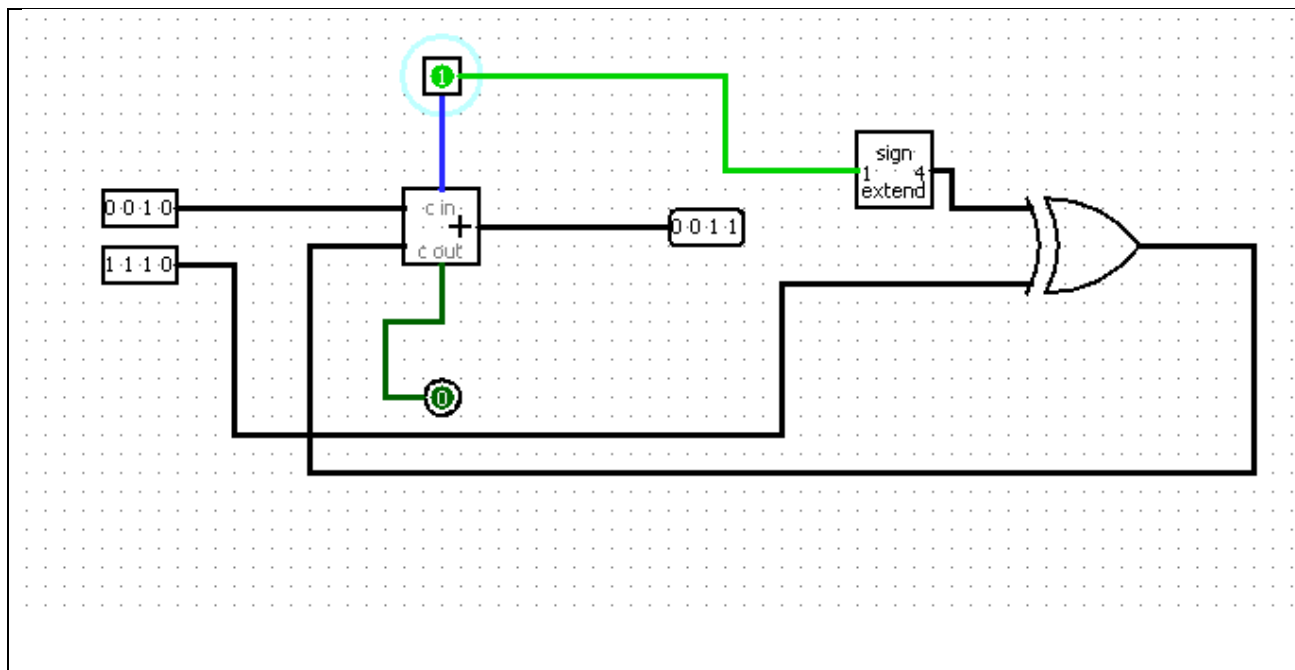
4-bit Adder (Gate Level):

Wave - Default		Msgs				
+ /tb_adder/x + /tb_adder/y + /tb_adder/sum + /tb_adder/cout	1111		1001	0000	1111	0110
	0010		1010	1001	0010	0001
	0001		0011	1001	0001	0111
	St1					

4-bit Adder (Behavioral Level):

Wave - Default		Msgs				
+ /tb_adder/x + /tb_adder/y + /tb_adder/ctrl + /tb_adder/sum	1001		1001	0000	1111	0110
	1010		1010	1001	0010	0001
	0					
	0011		0011	0111	1101	0111

Logisim (4-bit adder):



Task 2:

Description:

In this task, we were asked to make the 2X1 MUX by AND the two-bit input at D0 and OR the input to D1, so when selection line (s0) is 0 then the output of AND gate will appear on Y and when s0=1 then output of OR gate will appear on Y. In behavioral level, we implement this by using if-else and the Logisim circuit is implemented by taking the AND, OR gates and one MUX and input/output ports.

Code:

MUX_2X1 (Behavioral Level):

```
module MUX(A,B,s0,Y);
```

```
input A,B,s0;
```

```
output reg Y;
```

```
always @ (*)
```

```
begin
```

```
if (s0==0)
```

```
Y= A&B;
```

```
else

Y= A | B;

end

endmodule

module tb_mux_2x1();

reg a,b,s0;

wire y;

MUX m1(a,b,s0,y);

initial

begin

s0=0; a=0; b=0;

#50

s0=1; a=0; b=1;

#50

s0=0; a=1; b=0;

#50

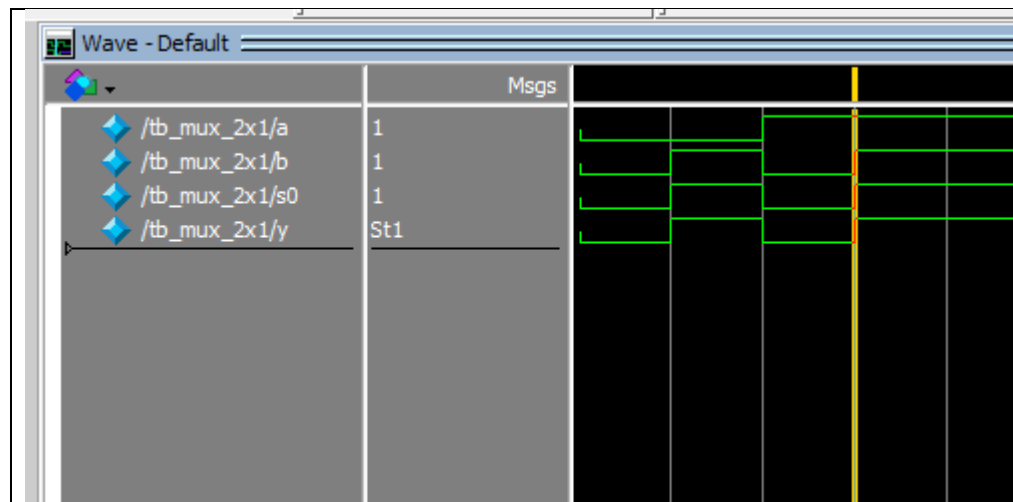
s0=1; a=1; b=1;

end

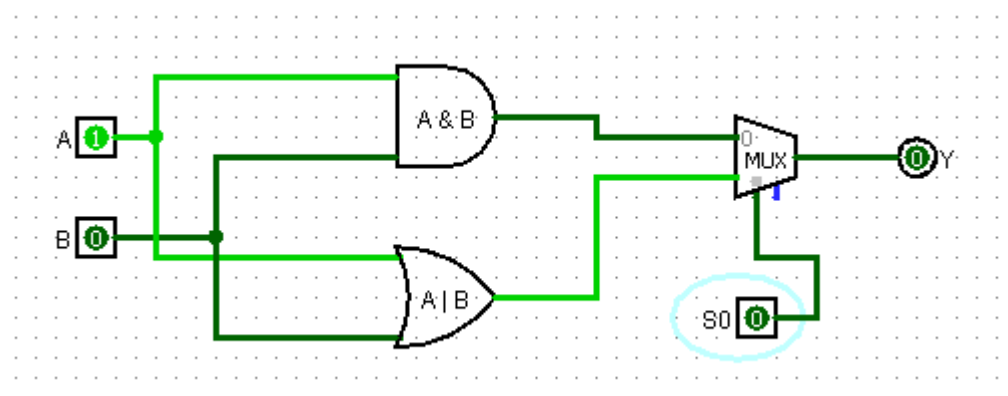
endmodule
```

Output:

MUX_2X1 (Behavioral Level):



Logisim (MUX_2X1):



Task 3:

Description:

In this task, we were asked to make the 2-bit comparator so for that we use Behavioral Level modeling and implement this by using if-else as if A equals B then only eq input is 1 and if A is greater than B then gt input is on and if A is less than B then lt input is on. The Logisim circuit is implemented by taking the comparator with two-bit inputs in unsigned manner and verifying the output for different inputs.

Code:

2-bit Comparator (Behavioral Level):

```
module comparator(A,B,eq,gt,lt);
input [1:0] A,B;
```

```
output reg eq,gt,lt;

always @(*)

begin

if(A==B)

begin

eq=1; gt=0; lt=0;

end

else if(A > B)

begin

eq=0; gt=1; lt=0;

end

else

begin

eq=0; gt=0; lt=1;

end

end

endmodule

module tb_comparator();

reg [1:0] a,b;

wire eq,gt,lt;

comparator uut(a,b,eq,gt,lt);

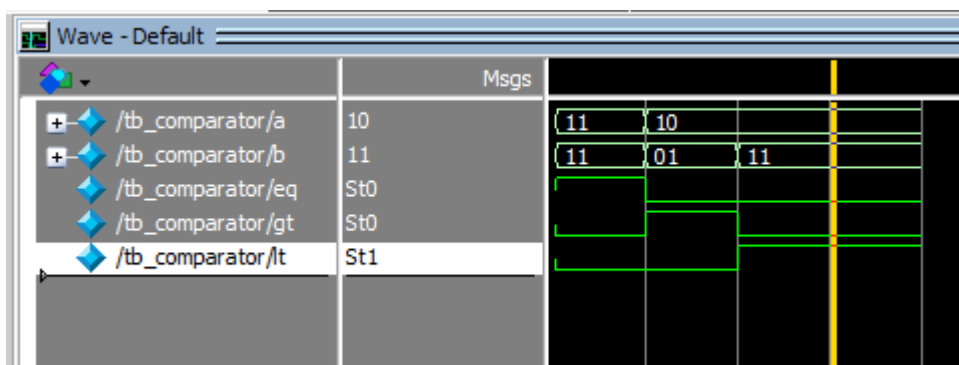
initial

begin
```

```
a=2'b11; b=2'b11; //eq
#50
a=2'b10; b= 2'b01; //gt
#50
a= 2'b10; b=2'b11; //lt
end
endmodule
```

Output:

2-bit Comparator (Behavioral Level):



Logisim (Comparator):

